



INSTITUTO CATÓLICO KAROL WOJTYLA

PROYECTO INTEGRADOR FERIA DE MICROCONTROLADORES ICKW 2026

Perfil del proyecto:

SafeSchool Alert: Sistema Inteligente de Alertas y Emergencias Escolares

Grado:

Tercer Año de Bachillerato Técnico en Desarrollo de Software

MÓDULO 3.1: Desarrollo de aplicaciones de software para la
solución de problemas

Alumnos:

José Rolando Velasco Peña

Luis Mario Meléndez Escobar

Ulises de Jesús Mercado Arberto

Docente:

Lic. Criseyda Guadalupe Araujo Mengar

Cabañas Oeste, El Salvador

10 de julio de 2026

INDECE

TABLA DE FIGURAS	3
1. INTRODUCCIÓN	4
2. DISEÑO DEL CIRCUITO ELECTRONICO	5
2.1. Componentes utilizados	5
2.2. Explicación del circuito	6
3. PROGRMACIÓN DEL ARDUINO	8
3.1. Variables principales	8
3.2. Enumeraciones (Enums)	10
3.3. Máquina de estados	11
3.4. Parser	14
6. COMUNICACIÓN SERIAL	15
6.1. Protocolo de Comunicación	15
6.2. Comandos de comunicación:	16
7. PRUEBAS REALIZADAS	17
7.1. Tabla de Pruebas Realizadas.	17
7.2. Capturas de evidencias de pruebas realizadas	18
8. PRBLEMAS ENCONTRADOS	21
8.1. Problemas Encontrados durante el desarrollo del circuito	21

TABLA DE FIGURAS

Figura 1 Diseño Final del circuito diseñado en Tinkercad de SafeSchool Alert.....	6
Figura 2 Interfaz del circuito al estar en estado seguro (Sin emergencias).....	12
Figura 4 Interfaz del circuito al estar en una emergencia activa por Incendio.....	12
Figura 5 Interfaz del circuito al cambiar el estado de una emergencia a en proceso.	13
Figura 6 Interfaz del circuito al cambiar el estado de una emergencia a Atendida.....	13
Figura 7 Interfaz del circuito al cambiar el estado de una emergencia a Falsa Alarma.....	14
Figura 8 Interfaz evidencia de prueba funcional de activación de emergencias	19
Figura 9 Interfaz evidencia de prueba funcional de validación de comunicación serial.....	19
Figura 10 Interfaz evidencia de prueba funcional de activación sirena de emergencia.....	20
Figura 11 Interfaz evidencia de prueba funcional del funcionamiento del sensor de MQ-2.....	20
Figura 12 Interfaz evidencia de prueba funcional, para mostrar mensajes dinámicos en LCD. .	21

1. INTRODUCCIÓN

En el presente documento se desarrolla la propuesta de solución para el problema planteado durante la etapa de análisis y diseño del proyecto. El objetivo de esta segunda etapa consiste en construir un prototipo funcional del circuito real del sistema en la plataforma Tinkercad, el cual simulará las funciones y el comportamiento principal del circuito que se implementará en el sistema SafeSchool Alert durante su fase final.

Gracias a la plataforma Tinkercad, se logró implementar y simular el funcionamiento principal del sistema. El circuito funciona como una máquina de estados propia de los sistemas embebidos. Su funcionamiento se basa en estados previamente definidos, tipos de emergencia y zonas. Cuando se activa una emergencia, ya sea de forma manual mediante los pulsadores o de forma automática a través del sensor de gas, el Arduino interpreta los comandos enviados por la aplicación móvil y los valores obtenidos del sensor para tomar decisiones y ejecutar acciones específicas, como activar la sirena de emergencia, encender los LED indicadores de estado y mostrar mensajes correspondientes a la emergencia detectada en la pantalla LCD.

En el desarrollo de este documento se ha documentado la arquitectura y la lógica del sistema diseñado en Tinkercad, la organización del código, las variables principales utilizadas, los *enum* empleados para representar los estados, los tipos de emergencia y las zonas, así como las funciones principales, la recepción de comandos, el protocolo de comunicación y la gestión de estados.

2. DISEÑO DEL CIRCUITO ELECTRONICO

2.1. Componentes utilizados

En este apartado se presentan los distintos componentes importantes y empleados para construir el circuito, como la cantidad de componentes, función e importancia de cada componente dentro del sistema, tales como sensores, LEDs, Resistencias, pantallas LCD entre otros. Los cuales han sido conectados correctamente con los pines analógicos y digitales del Arduino.

Tabla de componentes				
Nº	Componente	Función	Cantidad	Valor
1	Arduino	Microcontrolador en cargado de compilar el código y tomar decisiones en el circuito y activación de componentes (El cerebro del circuito)	1	--
2	Sensor MQ-2	Se encarga de detectar umbrales peligros de humo, tiene como propósito detectar de forma automática posibles incendios por presencia de humo.	1	--
3	Pantalla LCD	Mostrar mensajes de estado e información de emergencias activas o detectadas en la institución.	1	--
4	Potenciómetro	Su función en el circuito es ajustar el contraste, de la pantalla LCD para que esta muestre una imagen con mejor resolución.	1	4V
5	LED Rojo	Indica una emergencia activa detectada.	1	--
6	LED Verde	Indica que la emergencia activa fue atendida	1	--

7	LED Amarillo	Indica que la emergencia activa está en proceso	1	--
8	LED Azul	Indica que la emergencia activa, es falsa.	1	--
9	Pulsadores	Activan una emergencia manualmente, ya sea Médica o de Seguridad.	6	--
10	Buzzer	Emite sonido de una sirena de emergencia, en dos estados continua o intermitente.	1	--
11	10 kΩ Resistencia	Limita el flujo de corriente que pasa por el circuito para proteger los componentes.	1	10 kΩ
12	220 Ω Resistencia	Limita el flujo de corriente que pasa por el circuito para proteger los componentes.	12	220 Ω

Imagen del circuito:

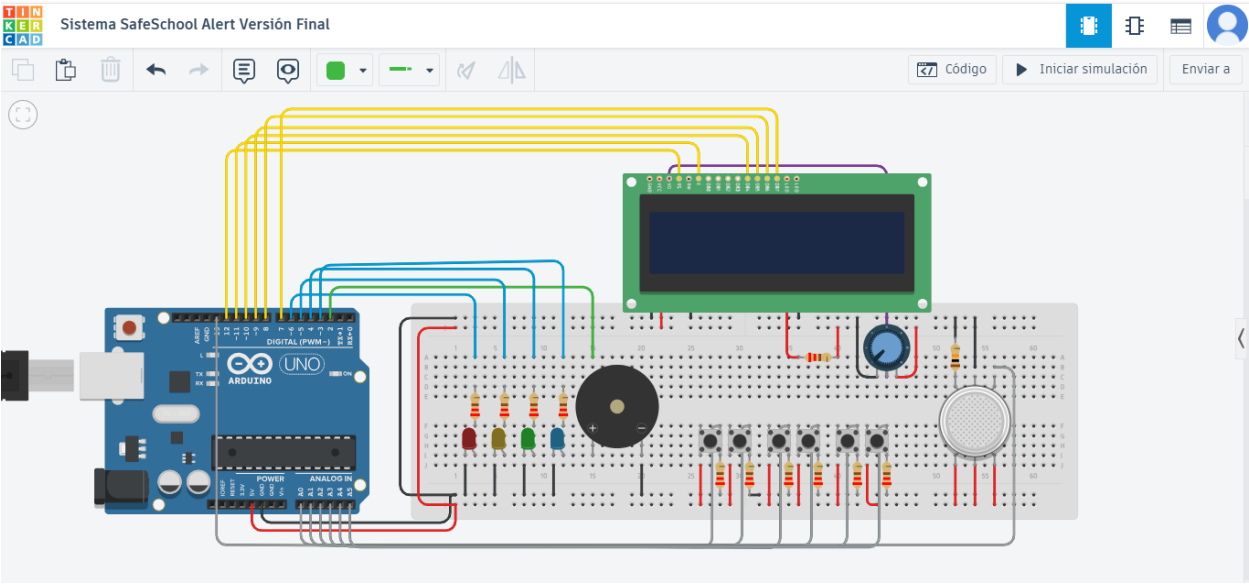


Figura 1 Diseño Final del circuito diseñado en Tinkercad de SafeSchool Alert.

2.2. Explicación del circuito

El circuito está conectado de forma estratégica y como lo amerita el componente, tomando en cuenta sus pines y si este tiene polaridad o no. Como bien se sabe los componentes

traen pines, donde estos se deben conectar de manera correcta y no como por pura su posición.

En esta sección se explica la conexión de cada componente con el Arduino y el circuito:

- **LEDs:** Se encuentran de la siguiente manera: Cátodo está conectado a tierra (GND) por donde la corriente regresa y pin Ánodo se encuentra conectado, con una resistencia de $220\ \Omega$ para limitar la corriente para proteger el componente, que está conectada a un Pin digital del Arduino.
- **Buzzer:** Este componente al tener polaridad, se conectó de la siguiente manera, el pin negativo a GND y el pin positivo del buzzer a un pin digital, con su respectiva resistencia de $220\ \Omega$, para limitar el flujo de la corriente.
- **Pulsadores:** Los pulsadores, para evitar lecturas confusas y estados disparados, se han conectados con el método: Pull-down, con esta forma el pin lee 0 hasta ser presionado que pasa a ser 1, además de tener una resistencia que va a GND en la misma dirección del pin digital.
- **Pantalla LCD:** La pantalla fue conectada de forma estratégica, se conectó a 4 pines digitales de 8 bits para ahorrar pines del Arduino. Las conexiones fueron de la siguiente manera:

Conexión de LCD 16x2			
N°	PIN	Conexión con Arduino (Pin)	Descripción
1	GND/VSS	GND	Es el pin a tierra de la pantalla
2	VCC/CDD	5V	Alimenta la pantalla
3	VO	Conexión con un potenciómetro	Controla el contraste, se conecta al pin central del potenciómetro
4	RS	Pin analógico	Es el pin de control, ayuda a decirle al LCD si le estás mandando comando o dato/texto
5	RW	GND	

6	E (Enable)	Pin analógico	Es el pin de habilitación, sirve para decirle al LCD “lee ahora lo que te estoy enviando”
7	DB0 a DB7	Pin analógico	Son pines de datos. Se usa el LCD en modo de 4 bits, no de 8.
8	LED 1 Anodo (+)	5V	Este método sirve para darle mayor resolución a la pantalla.
9	LED 2 Cátodo (-)	GND	Conectado a GND del Arduino.
10	Resistencia	220 ohmios	Esta va conectada a LED 1 Anodo, para limitar la corriente que pasa, por la LCD.

- **Sensor MQ-2:** Este se encuentra conectado de la siguiente manera: Terminal A1 se encuentra conectado con GND con una resistencia de 10 k Ω , y A2 se encuentra conectado a un pin Analógico para poder leer las lecturas del sensor de mejor manera, y B1, B2 y H2 se encuentran conectado a V5.

3. PROGRAMACIÓN DEL ARDUINO

3.1. Variables principales

3.1.1. Variables de control de tiempo: Estas variables almacenan el último momento en que se realizó un cambio de estado en los diferentes componentes del sistema, utilizando la función millis(). Su propósito es controlar los intervalos de tiempo de los LEDs, la sirena y el modo seguro, permitiendo que funcionen sin detener la ejecución del programa y logrando que todas las tareas se ejecuten de manera simultánea.

```
unsigned long ultimoCambioLedRoj = 0;
unsigned long ultimoCambioLedAma = 0;
unsigned long ultimoCambioLedVer = 0;
unsigned long ultimoCambioLedAzu = 0;
unsigned long ultimoCambioSeguro = 0;
unsigned long ultimoCambioSirenaCT = 0;
unsigned long ultimoCambioSirenaDT = 0;
```


3.1.2. Variables de pines y componentes: Estas variables almacenan la asignación de los pines digitales y analógicos del Arduino, utilizados por cada componente. El propósito de estas variables es identificar de forma clara las conexiones de los dispositivos, como LEDs, Pusadores, Buzzer y Sensor de Gaz.

```
int led_Rojo = 6;
int led_Amari = 5;
int led_Verde = 4;
int led_Azul = 3;

int btn_Edi_A_Medica = 13;
int btn_Edi_A_Seguridad = A1;

int btn_Edi_B_Medica = A2;
int btn_Edi_B_Seguridad = A3;

int btn_Edi_C_Medica = A4;
int btn_Edi_C_Seguridad = A5;

int buzzer = 2;
int sensor = A0;
```

3.1.3. Constantes del sistema

Estas constantes almacenan valores fijos utilizados dentro del programa tales como. TAM_BUFFER define el tamaño máximo del buffer de recepción de datos (Comandos que envía la app), mientras que UMBRAL_SEGURO establece el valor límite del sensor de gas para determinar si existe una situación de riesgo.

```
const byte TAM_BUFFER = 30;
const int UNBRAL_SEGURO = 500;
```

3.1.4. Variables de recepción de datos: Estas variables son las encargadas de almacenar temporalmente los comandos enviados por serial y establecer un índice dentro del buffer. Estas son “buffer, indice”. Ambas trabajan en conjunto, buffer almacena una lista de caracteres de 30

espacios, donde índice es el encargado de decirle al sistema en que índice guardar, cada carácter que llega por serial.

```
char buffer[TAM_BUFFER];  
int indice = 0;
```

3.1.5. Variables de estado: Estas variables de tipo bool se utilizan para controlar diferentes condiciones del sistema, indicando si una acción o un estado se encuentra activo (true) o inactivo (false). Un ejemplo de estas variables es

```
bool estadoHumo = false;  
bool yaCambioASeguro = false;
```

3.2. Enumeraciones (Enums)

Los enum, también llamados enumeraciones, son un tipo de dato que permite asignar un nombre lógico a un conjunto de valores enteros. En lugar de trabajar con números difíciles de interpretar, como 0, 1 o 2, se pueden utilizar nombres más descriptivos, como EstadoSeguro, Incendio o EmergenciaMedica, haciendo que el código sea más fácil de leer y mantener.

Por defecto, las enumeraciones comienzan en 0, aunque es posible indicar el valor inicial de la primera enumeración. A partir de ese valor, las demás continúan la secuencia automáticamente.

En el sistema SafeSchool Alert, los enum son fundamentales, ya que permiten organizar la lógica del programa de una forma clara. Se utilizan para representar los diferentes tipos de emergencia, controlar el estado de la recepción de comandos enviados por comunicación serial (por ejemplo, si el mensaje se está recibiendo, ya fue recibido o ocurrió un error) y almacenar las diferentes zonas de la institución, como Edificio A, Edificio B y Edificio C.

Enums del Sistema:

- **Enum de estados de recibimiento de datos:** Este enum se encarga de declarar los distintos tipos de estados de recibimiento de los comandos enviados por serial, los cuales

ayudan a entender cuando un mensaje ya llegó completo, u ocurrió un error al recibir datos.

```
enum estadoRecepcion {  
    RECIBIENDO,  
    MENSAJE_COMPLETO,  
    ERROR_BUFFER,  
};
```

- **Enum de estados del sistema:** Este es el más importante de todo el sistema por que se encarga de controlar los distintos estados que pueden, existir durante una emergencia en el sistema:

```
enum estadoEmergencia {  
    SEGURO,  
    EMERGENCIA_ACTIVA,  
    INCENDIO_ATENDIDA,  
    MEDICA_PROCESO,  
    MEDICA_ATENDIDA,  
    MEDICA_FALSA,  
    SEGURIDAD_PROCESO,  
    SEGURIDAD_ATENDIDA,  
    SEGURIDAD_FALSA,  
};
```

- **Enum de tipos de emergencia:** Se encarga de establecer cuáles son las diferentes emergencias que el sistema puede detectar y controlar sus estados.

```
enum tipoEmergencia{  
    TE_NINGUNA,  
    INCENDIO,  
    MEDICA,  
    SEGURIDAD,  
};
```

3.3. Máquina de estados

En esta sección se desarrolla cuáles son los distintos estados, que una emergencia puede pasar, según las acciones que el usuario realice desde la aplicación móvil, el sistema controla 5

estados de emergencia, es decir al activarse una emergencia ya sea por detección de un sensor o por medio de los pulsadores, que son las siguientes:

- **Sin emergencias:** Es el estado seguro del instituto cuando, no existe o sea detectado una emergencia de ningún tipo. En el sistema se identifica este estado como: “SEGURO”.

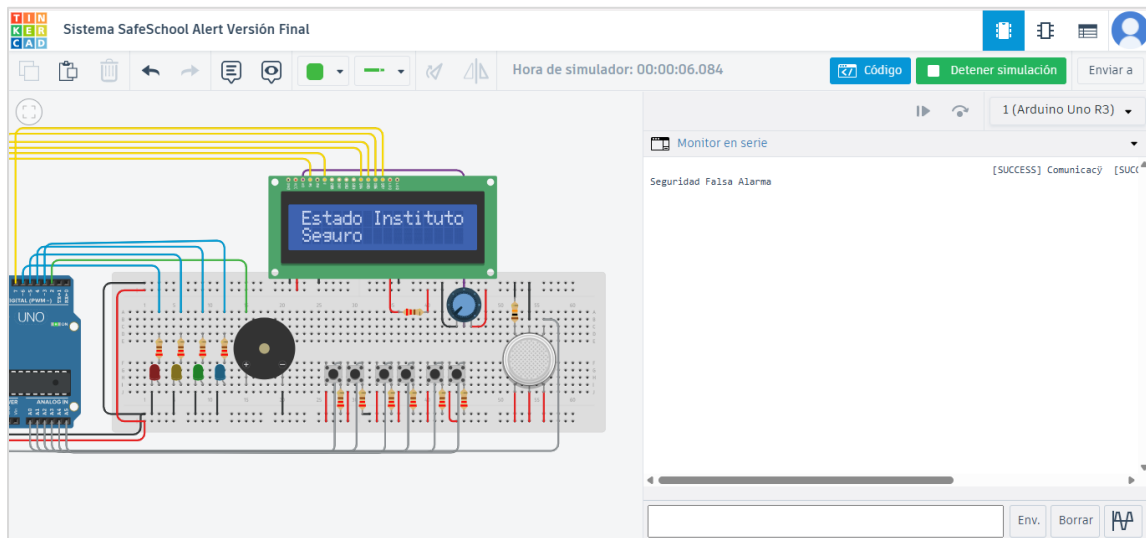


Figura 2 3 Interfaz del circuito al estar en estado seguro (Sin emergencias).

- **Emergencia activa:** Es cuando se detecta una emergencia, ya se provocada por un incendio, médica o de seguridad. En el sistema se identifica este estado como: “EMERGENCIA_ACTIVA”.

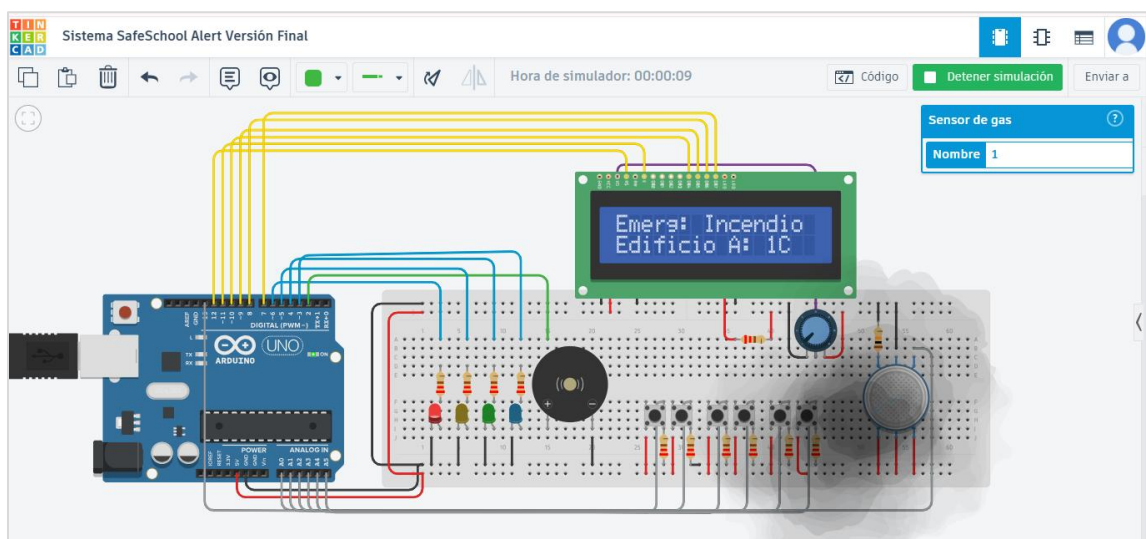


Figura 4 Interfaz del circuito al estar en una emergencia activa por Incendio

- **Emergencia en proceso:** Este estado significa que la emergencia fue detectada y en ese momento se está atendiendo, para darle una solución, esta puede ser de tipo médica o de seguridad. En el sistema identifica este estado como: “TIPO_PROCESO”.

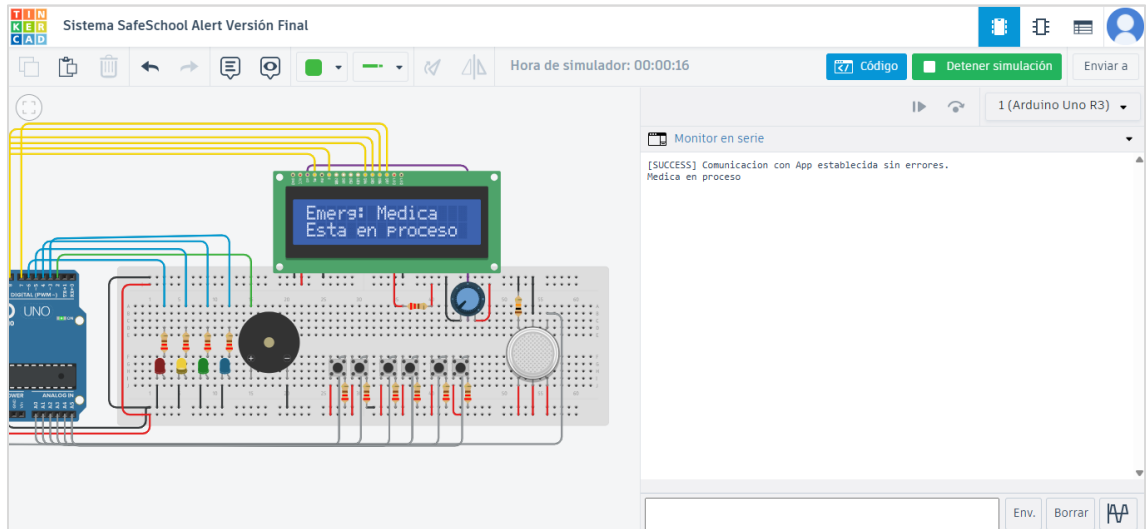


Figura 5 Interfaz del circuito al cambiar el estado de una emergencia a en proceso.

- **Emergencia atendida:** El significado de este estado es, que la emergencia activa dentro del sistema, fue atendida exitosamente, donde el personal capacitado a tratado con dicha emergencia. En el sistema se identifica este estado como: “TIPO_ATENDIDA”.

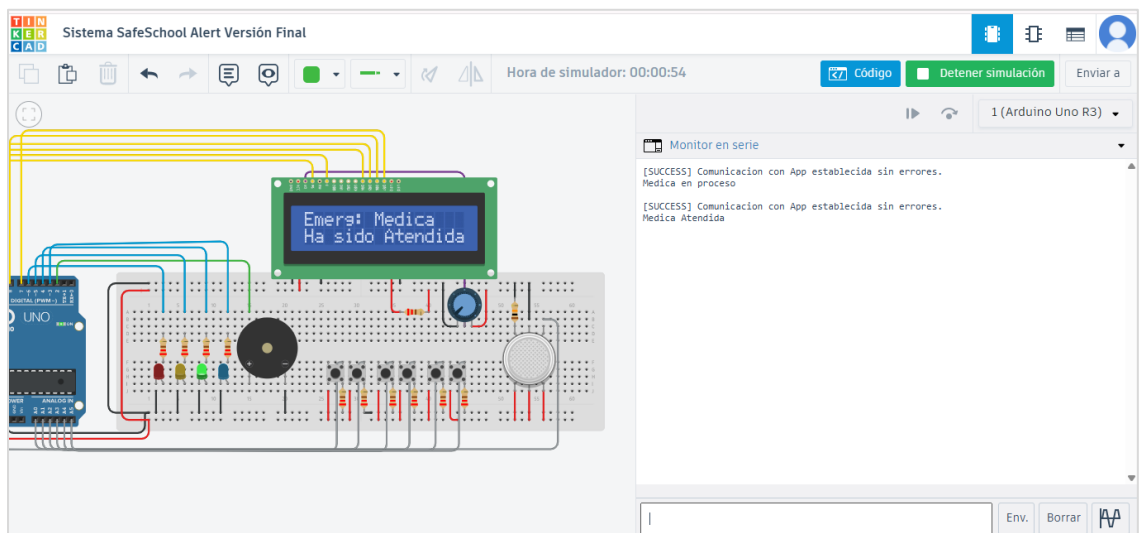


Figura 6 Interfaz del circuito al cambiar el estado de una emergencia a Atendida

- **Emergencia falsa:** Este estado aparece cuando, el usuario selecciona que la emergencia activa es una falsa alarma, lo que indica que el estado del instituto es seguro. En el sistema se identifica este estado como: “TIPO_FALSA”.

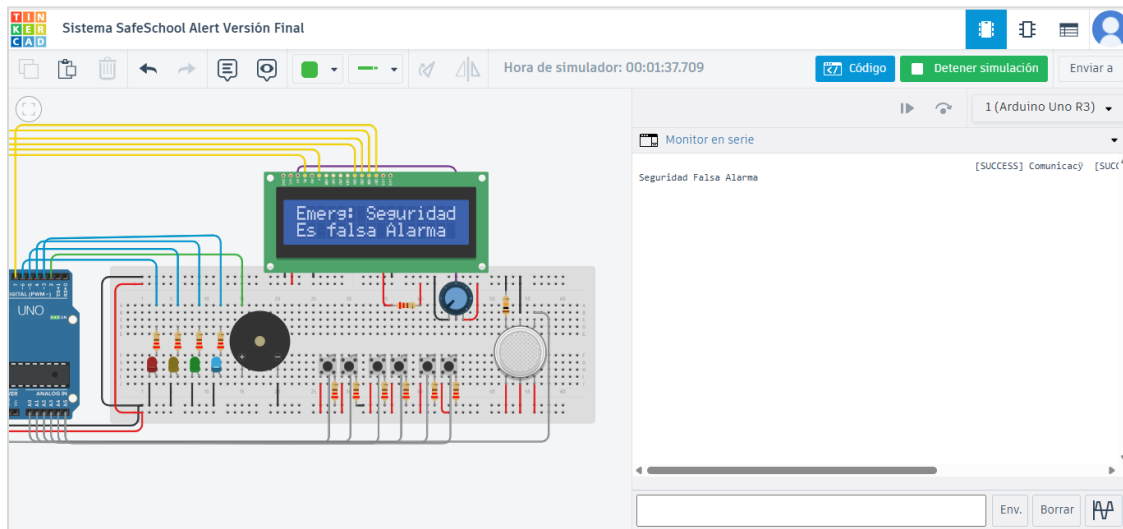


Figura 7 Interfaz del circuito al cambiar el estado de una emergencia a Falsa Alarma

3.4. Parser

El parser dentro del sistema es el encargado de separar e interpretar todos los datos que llegan por comunicación serial desde la aplicación móvil. Su función es reconocer qué comandos son válidos y aceptados por el sistema y cuáles no. Por ejemplo, si la aplicación envía un mensaje como "Medica esta Atendida", este no cumple con el protocolo de comunicación y el sistema genera un error de tipo [COMMAND ERROR]. Esto se debe a que el sistema solo reconoce comandos con el formato Tipo,Estado#, como, por ejemplo: Medica,Atendida#.

Cuando un comando con el formato correcto llega por comunicación serial, el sistema lo interpreta y actualiza el estado correspondiente de la emergencia. En este caso, el estado de la emergencia médica cambia a Atendida. La función encargada de realizar este proceso es descifrarEstados(String comando).

```
void descifrarEstados(String comando) {  
  
}
```

6. COMUNICACIÓN SERIAL

La comunicación serial es una de los temas más importantes dentro del sistema por que la comunicación serial es la encargada de servir como puente de comunicación entre la aplicación y el Arduino, por lo que conocer, cómo funciona y se maneja la comunicación es fundamental. Atentes de comenzar a fondo con este tema, cabe recalcar que el circuito desarrollado en esta segunda etapa, se desarrolló y diseño dentro de la plataforma de Tinkercard.

Dentro de este simulador que nos permite crear circuitos dinámicos, no se encuentra el Módulo de Bluetooth, por lo que para enviar los mensajes a través de serial se hace a través del input de serial de Tinkercard, en este campo se deben escribir los comandos manualmente, que ya en la implantación en físico la aplicación enviara a través de bluetooth.

6.1. Protocolo de Comunicación

La comunicación y recepción de datos por serial se realiza a través de la función `leerSerial()`. Dentro de esta función se desarrolla toda la lógica para capturar los datos que vienen desde el puerto serial. Para poder recolectar los datos, se diseñó y creó un nuevo *buffer* que funciona de forma similar al que tiene el Arduino Uno por defecto para recibir datos. Se decidió crear uno propio porque la comunicación serial detenía el sistema durante 1 segundo, ya que esperaba a recibir todos los caracteres enviados por serial antes de continuar con la ejecución, lo que ocasionaba errores de ralentización.

El nuevo *buffer* funciona de forma similar, solo que, en vez de detener el sistema, lo que hace es que va recolectando carácter por carácter en cada vuelta del `loop()`, almacenándolos en la variable `buffer`, que tiene una capacidad máxima de 30 caracteres. El sistema sabe dónde termina la cadena de caracteres del comando cuando recibe el símbolo de numeral (`#`). Este carácter le indica al sistema que ahí termina el comando. Luego, el sistema guarda el comando

y lo envía a la función `descifrarEstados()`, que se encarga de reconocer e interpretar los comandos recibidos.

6.2. Comandos de comunicación:

Todos los mensajes enviados entre la aplicación y Arduino siguen un formato establecido. Si la aplicación envía un comando que no cumple con el protocolo de comunicación, el Arduino ignorará todos los comandos enviados que no cumplan el formato establecido y mostrará un error de tipo `[COMMAND ERROR]`, en serial. De igual manera, el sistema solo permite comandos de un máximo de 30 caracteres; si el comando sobrepasa el límite de caracteres, el Arduino ignorará el comando y mostrará un error de tipo `[COMMAND ERROR]` en serial.

- **Cambiar estado de emergencia Médica a Atendida:**

```
Medica,Atendida#
```

- **Cambiar estado de emergencia Médica a En Proceso:**

```
Medica,EnProceso#
```

- **Cambiar estado de emergencia Médica a Falsa Alarma:**

```
Medica,Falsa#
```

- **Cambiar estado de emergencia de Seguridad a Atendida:**

```
Seguridad,Atendida#
```

- **Cambiar estado de emergencia de Seguridad a En Proceso:**

```
Seguridad,EnProceso#
```


- **Cambiar estado de emergencia de Seguridad a Falsa:**

Seguridad,Falsa#

7. PRUEBAS REALIZADAS

7.1. Tabla de Pruebas Realizadas.

Para asegurar que el sistema cumpla con los requisitos establecidos y ejecute correctamente las tareas programadas, sin presentar errores en la lógica de funcionamiento ni en la gestión de los estados de emergencia, en su versión final y optimizada que será la entregada en este avance, se realizaron múltiples pruebas funcionales mientras el sistema se encontraba en operación y atendía una emergencia activa de cualquier tipo. El objetivo de estas pruebas fue identificar fallas, detectar errores y proponer una solución para cada uno de ellos.

Para representar las pruebas realizadas al circuito electrónico basado en Arduino y al programa desarrollado en C++ dentro del simulador Tinkercad, se diseñó la siguiente tabla, en la cual se muestran las diferentes pruebas efectuadas y los resultados obtenidos durante la ejecución de cada prueba funcional.

Pruebas Funcionales del Circuito Electrónico (Versión Final)							
ID	Módulo	Caso de Prueba	Entrada	Resultado esperado	Resultado	Estado	Evidencia
1	Módulo de activación de emergencias (Pulsadores)	Comprobar que el circuito, lea correctamente el estado del pulsador para activar una emergencia.	Clic en el pulsador de emergencia médica, Edificio A	El circuito activa una emergencia médica en el edificio A del instituto.	El circuito, activo la emergencia médica, y mostro la zona afectad.	PASS	Captura-01
2	Módulo de comunicación Serial	Verificar que el circuito reciba los comandos	Escribir el input Serial: Medica, Atendida#	El circuito cambia el estado de la	El circuito recibe el comando correctamente y	PASS	Captura-02

		enviados correctamente, por serial.		emergencia médica a Atendida, en serial	da una respuesta en serial.		
3	Módulo de activación de sirenas de emergencias.	Verificar que el circuito active la sirena continua, al momento de activarse una emergencia.	Activación de emergencia de seguridad	El circuito, enciende el LED de color rojo, y activa el buzzer a sirena continua.	El circuito, activa correctamente el LED rojo y el buzzer pasa a sirena continua.	PASS	Captura-03
4	Módulo de sensores (MQ-2)	Verificar que el sensor de humo detecta umbrales peligrosos de humo mayores a 400.	Simulación de humo presente en el sensor.	El circuito debe pasar a estado de emergencia activa y encender sirena de emergencia	El circuito detecta el umbral mayor a 400 y activa la emergencia por incendio.	PASS	Captura-04
5	Módulo de mensajes LCD	Determinar si el circuito muestra mensajes dinámicos de estados de emergencia en LCD.	Activa una emergencia de seguridad en el Edificio B.	El circuito debe mostrar el mensaje de emergencia, en la ubicación: Edificio B.	El circuito muestra correctamente el mensaje de emergencia en la pantalla LCD.	PASS	Captura-05

7.2. Capturas de evidencias de pruebas realizadas

7.2.1. Captura-01: Evidencia de la prueba funcional del módulo de activación de emergencias correspondiente al caso de prueba: Validación de la activación manual de emergencias mediante pulsadores.

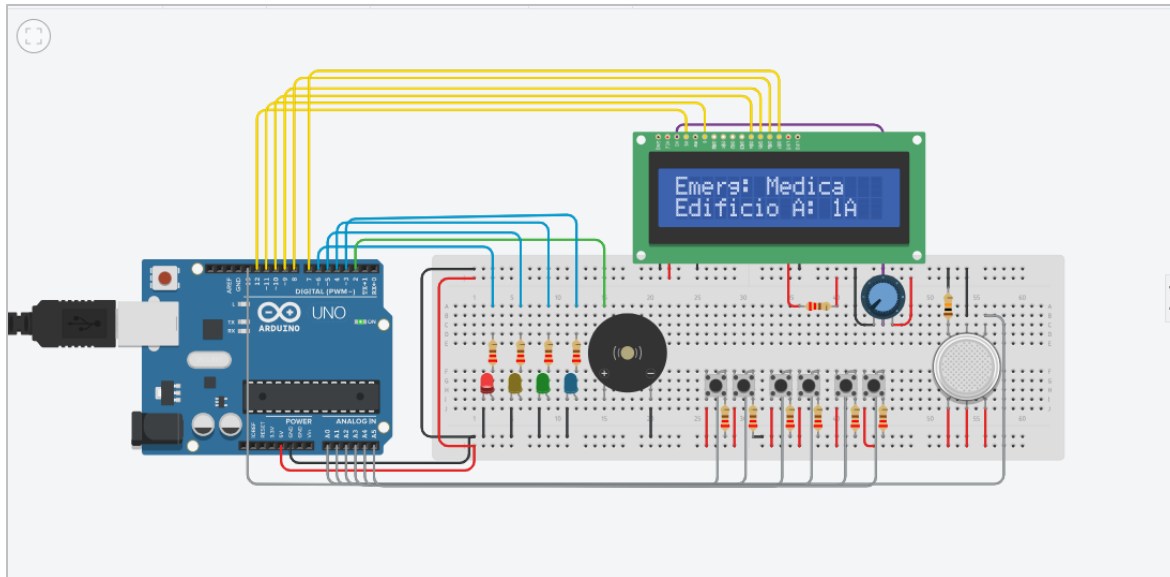


Figura 8 Interfaz evidencia de prueba funcional de activación de emergencias

7.2.2. Captura-02: Evidencia de la prueba funcional del módulo de comunicación serial, que corresponde al caso de prueba: Verificación de recibimiento de datos por medio de serial, enviados por serial.

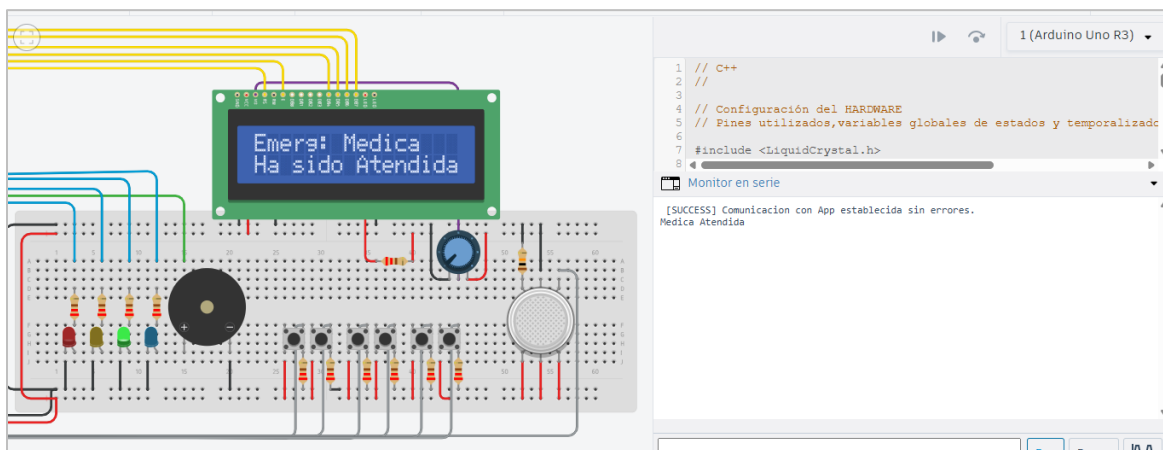


Figura 9 Interfaz evidencia de prueba funcional de validación de comunicación serial.

7.2.3. Captura-03: Evidencia de prueba funcional correspondiente al módulo de activación de sirenas de emergencias, para el caso de prueba: Activación de la sirena continua, al momento de activarse una emergencia.

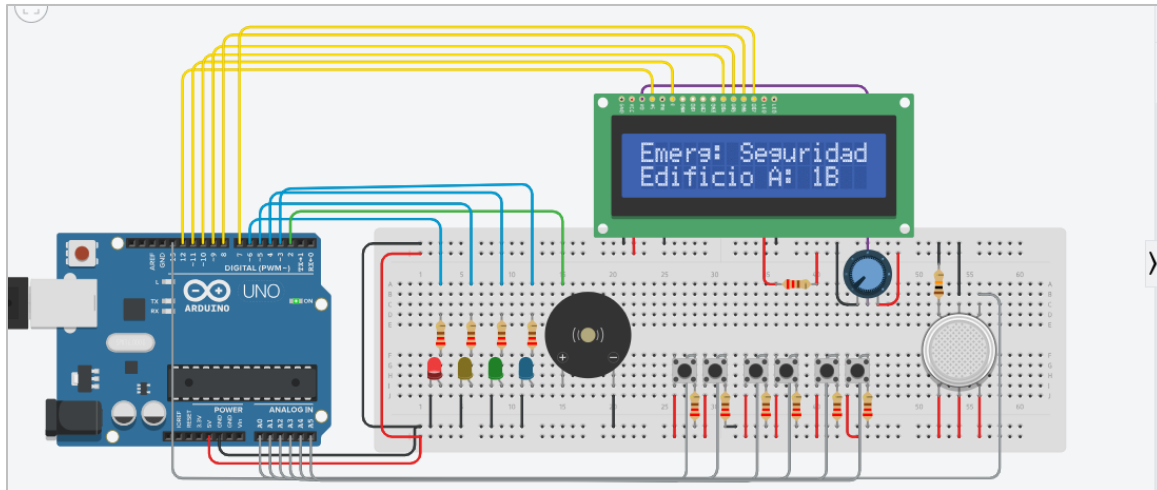


Figura 10 Interfaz evidencia de prueba funcional de activación sirena de emergencia.

7.2.4. Captura-04: Evidencia de prueba funcional correspondiente al módulo de sensores (MQ-2), para el caso de prueba: Detección de umbrales peligrosos por presencia de humo e indicios de incendio.

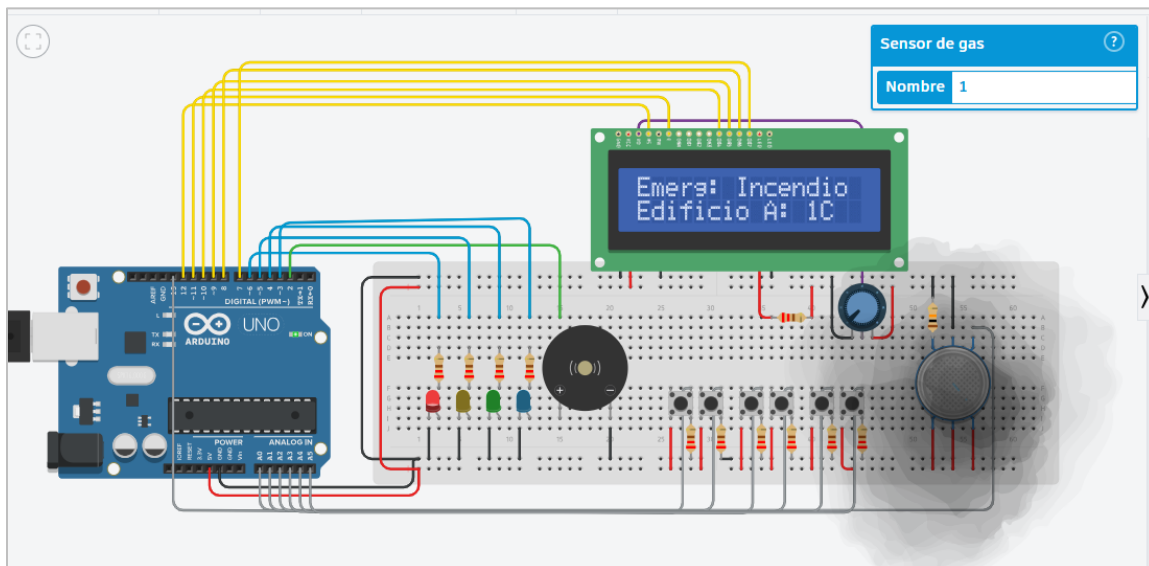


Figura 11 Interfaz evidencia de prueba funcional del funcionamiento del sensor de MQ-2

7.2.5. Captura-05: Evidencia de prueba funcional correspondiente al módulo de mensajes LCD, para el caso de prueba: Activación de emergencia de seguridad en el edificio B y mensajes dinámicos de estados en pantalla LCD de la emergencia de seguridad.

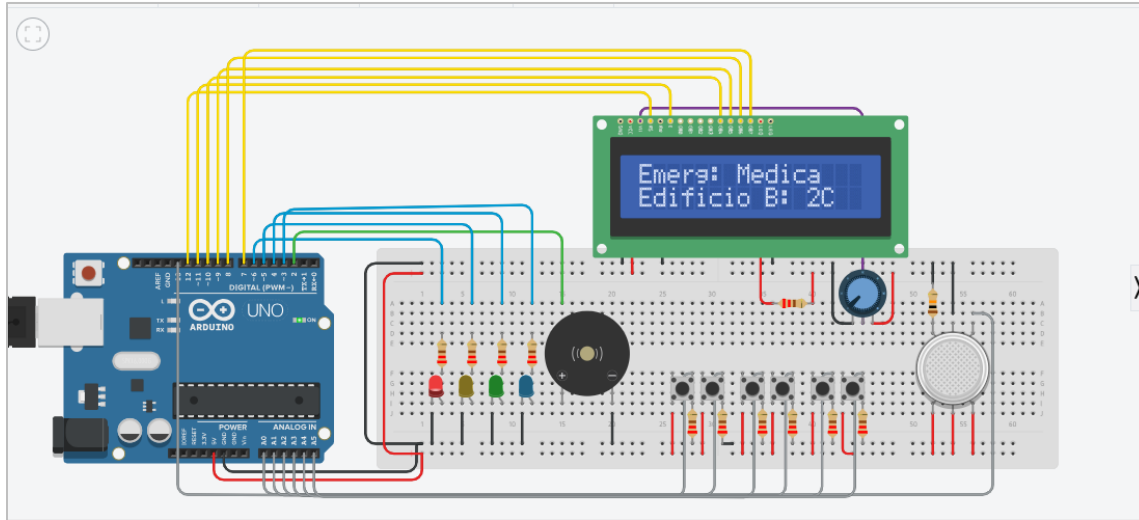


Figura 12 Interfaz evidencia de prueba funcional, para mostrar mensajes dinámicos en LCD.

8. PRBLEMAS ENCONTRADOS

8.1. Problemas Encontrados durante el desarrollo del circuito

En este apartado se documentan las principales fallas y errores, tanto de lógica como de sintaxis, que fueron identificados durante el desarrollo y diseño del circuito. Es importante destacar que los errores descritos corresponden únicamente a las etapas de desarrollo del prototipo y no a la versión final del sistema, la cual fue sometida a las pruebas de funcionalidad presentadas en el apartado anterior.

Durante el proceso de desarrollo, el sistema pasó por tres versiones. Los problemas que se describen en esta sección corresponden a las versiones 1 y 2, ya que fue en estas donde se identificaron las principales deficiencias. Posteriormente, dichas fallas fueron corregidas y optimizadas en la versión 3, la cual incorpora una arquitectura mejorada y un funcionamiento más estable, convirtiéndose en la versión final entregada.

8.1.1. Problema 1: Ralentización del de ejecución por el uso de delay().

El sistema se ralentizaba y detenía durante 3 segundos cada vez que recibía comandos por serial y en ocasiones el sistema fallaba y quedaban estados de emergencia sueltos.

Causa: El problema era ocasionado por el uso excesivo de `delay()`, para esperar 1 segundo para apagar y encender un LED y para crear melodías en el buzzer para asemejarse a una sirena de emergencia real.

Solución: Se implemento el uso de `millis()`; para todas aquellas acciones que necesiten una espera antes de activarse, como para cambiar de estado de emergencia a seguro y se redujo la lógica innecesaria y el uso de `delay` para encender y apagar LEDs, de forma repetida que era el problema principal.

8.1.2. Problema 2: Ralentización de la comunicación serial y recibimiento de comandos

En la versión 2 del sistema se presentó una ralentización al momento de recibir los comandos por comunicación serial. Mientras el Arduino esperaba recibir el mensaje completo, la sirena y los LEDs permanecían en el mismo estado durante aproximadamente un segundo, lo que ocasionaba pequeños retrasos en la respuesta del sistema.

Causa:

Este problema ocurría porque el Arduino esperaba recibir el comando completo antes de continuar con la ejecución del programa. Mientras se recibían los caracteres del mensaje, el sistema se detenía por un momento, provocando un retraso en el funcionamiento de los demás componentes.

Solución:

Para solucionar este problema se creó un nuevo buffer de recepción, similar al que utiliza Arduino para recibir datos por serial. La diferencia es que este nuevo buffer va almacenando cada carácter recibido en cada vuelta de la función `loop()`, guardándolo en una variable de tipo `char` llamada `buffer`, declarada como `char buffer[30];`.

De esta manera, el sistema ya no se detiene mientras espera el mensaje completo. En cada vuelta del `loop()` se recibe un carácter y se almacena en la posición indicada por la variable `indice`, la cual aumenta conforme llegan nuevos datos. Gracias a esta mejora, se eliminó el retraso en la comunicación serial y el sistema respondió de forma más rápida y estable.

8.1.3. Problema 3: Uso excesivo de variables de tipo String

En las primeras versiones del sistema se utilizaban muchas variables de tipo String para guardar los estados del sistema, el tipo de emergencia, la ubicación y otros datos. Esto hacía que el programa consumiera más memoria de la necesaria y no estuviera bien optimizado.

Causa:

El problema se debía a que las variables de tipo String ocupan más memoria. Al utilizarlas para controlar casi todo el sistema, el Arduino consumía más recursos de los necesarios.

Solución:

Para mejorar el rendimiento del sistema, las variables de tipo String fueron reemplazadas por enum, los cuales permiten representar los estados mediante valores más simples. Se crearon enum para los estados del sistema, los tipos de emergencia, las zonas del instituto y la recepción de comandos. Gracias a este cambio, el código quedó más ordenado, fue más fácil de mantener y el consumo de memoria se redujo.